



BUNNIES & TEA

IRENE DIGES GARCÍA
2ºDAW

Contenido

1	Introducción	3
2	Definición del Proyecto	4
2.1	Descripción general del proyecto.....	4
2.2	Tipos de Empresas y Sectores Productivos	4
3	Diseño y Fases del Proyecto	5
3.1	Objetivos y Especificación de Requisitos.....	5
3.1.1	Requisitos funcionales	5
3.1.2	Requisitos no funcionales.....	6
3.2	Estudio de Viabilidad y DAFO	7
3.3	Herramientas de diseño y tecnologías	8
3.4	Documentación del Diseño	9
3.4.1	Diagrama de Casos de Uso	9
3.4.2	Modelo Entidad-Relación y Diccionario de Datos	9
3.4.3	Mockups e Interfaces	15
3.5	Fases del proyecto	18
3.6	Evaluación económica y financiación	19
4	Planificación del Proyecto	20
4.1	Secuenciación de actividades o tareas	20
4.2	Diagrama de secuenciación de proyecto	21
4.3	Procedimientos de ejecución	21
4.4	Prevención de riesgos y seguridad	22
5	Implementación y Codificación	24
5.1	Arquitectura del Sistema (Patrón MVC)	24
5.1.1	Diagrama de clases	25
5.2	Desarrollo backend.....	25

5.2.1	Consultas Atómicas como Solución.....	26
5.2.2	Transacciones Seguras (PDO) en la Tienda.....	26
5.3	Desarrollo Frontend.....	26
5.3.1	El problema de la latencia y el "Parpadeo de Red"	27
5.3.2	Predicción del Cliente y Descarte de Respuestas Obsoletas.....	27
6	Pruebas y validación del proyecto	28
6.1	Evaluación y seguimiento	28
6.2	Indicadores de calidad.....	29
6.3	Informe de evaluación de incidencias y soluciones	31
6.4	Gestión de cambios	31
7	Implantación del proyecto	32
7.1	Plan de implantación y despliegue.....	32
7.2	Manual de instalación	33
7.3	Pasos de instalación:	33
7.4	Manual de usuario.....	33
8	Conclusiones.....	35
8.1	Síntesis de resultados	35
8.2	Líneas futuras de trabajo.....	35
8.3	Conclusión personal	36
9	Bibliografía y Webgrafía	37

TÍTULO DEL PROYECTO: BUNNIES & TEA.

AUTOR: IRENE DIGES GARCÍA.

CURSO: 2º DAW, AÑO 25/26.

MÓDULO: PROYECTO INTERMODULAR II

GITHUB: [HTTPS://GITHUB.COM/CREEPYKAI/BAT](https://github.com/creepykai/BAT)

1 INTRODUCCIÓN

En este proyecto presento "Bunnies & Tea", una aplicación web que he desarrollado basándome en el género de videojuegos conocidos como Idle o Clicker. Básicamente, este tipo de juegos consisten en que el jugador empieza haciendo clics manualmente para conseguir recursos y, poco a poco, va invirtiendo esos beneficios para automatizar el proceso y generar ingresos de forma pasiva.

El objetivo principal que tenía en mente al crear "Bunnies & Tea" era hacer un simulador de gestión que fuera relajante, donde el usuario asume el papel de gerente de una cafetería temática.

Sin embargo, más allá de hacer un juego entretenido, mi verdadera motivación para este proyecto era retarme a nivel técnico. Quería ir un paso más allá de los requisitos básicos y aplicar un patrón de diseño MVC (Modelo-Vista-Controlador) puro creado desde cero. Además, me propuse el reto de implementar un sistema de comunicación asíncrona entre el cliente y el servidor en todo momento. De esta forma, el juego puede guardar el progreso y actualizar los datos en tiempo real sin tener que recargar la página nunca, logrando una experiencia fluida y mucho más profesional.

2 DEFINICIÓN DEL PROYECTO

2.1 DESCRIPCIÓN GENERAL DEL PROYECTO

A grandes rasgos, "Bunnies & Tea" es un juego de gestión económica interactivo. La idea principal es que el jugador acumule recursos (monedas y productos) de dos formas: activamente, haciendo clic en la pantalla, y pasivamente, dejando que el juego genere ingresos con el tiempo.

Conforme el jugador va ganando dinero, puede reinvertirlo en su cafetería. Por un lado, puede contratar personal (conejos empleados) que se encargan de automatizar la producción pasiva. Por otro lado, puede entrar a la tienda para comprar mejor maquinaria e infraestructuras que aumentan la eficiencia y la cantidad de recursos generados.

Además, he implementado un sistema de "prestigio" (que en el juego llamo "ascender" usando hojas de té). Esto permite al jugador reiniciar su partida perdiendo el dinero y los empleados actuales, pero a cambio consigue bonificaciones multiplicadoras permanentes. Es una mecánica muy típica en este género para mantener el interés a largo plazo y añadir una capa extra de estrategia.

2.2 TIPOS DE EMPRESAS Y SECTORES PRODUCTIVOS

La temática del juego simula el día a día de una empresa del Sector Terciario, concretamente enfocada en servicios de hostelería. Aunque el entorno sea lúdico, las mecánicas que he programado modelan procesos reales de cualquier negocio: el jugador tiene que gestionar sus finanzas (decidir dónde invertir para recuperar la inversión lo antes posible), controlar costes, y gestionar sus recursos humanos (contratando empleados con distintas características).

He diseñado este proyecto pensando en que la arquitectura que hay por debajo no se limite a un simple juego. De hecho, la lógica que he utilizado para estructurar la base de datos, gestionar el dinero de los usuarios y sincronizar datos en tiempo real podría adaptarse perfectamente a las necesidades de una empresa real. Se podría usar esta misma base para crear un software de gestión de inventario, controlar el stock de un almacén, o monitorizar la productividad y ventas de una tienda real.

Por otro lado, a nivel comercial y de producto, mi intención es que este proyecto no se quede solo en una práctica académica. Una vez que perfeccione las mecánicas y el código base, tengo planeado escalar el juego y empaquetarlo para llevarlo a plataformas móviles como Android o incluso publicarlo en Steam para PC.

3 DISEÑO Y FASES DEL PROYECTO

3.1 OBJETIVOS Y ESPECIFICACIÓN DE REQUISITOS

Al empezar el proyecto, mi objetivo principal era hacer una aplicación web que fuera rápida, que no diera tirones y que guardara la información de los jugadores sin fallos. Para lograrlo, dividí los requisitos en dos partes:

3.1.1 Requisitos funcionales

ID	Requisito	Especificación Concreta
RF-01	Gestión de Usuarios	El sistema debe permitir el registro de nuevos usuarios (email/password) y el inicio/cierre de sesión seguro mediante el manejo de variables de sesión (\$_SESSION).
RF-02	Generación Activa	Al interactuar con el elemento central de la interfaz, el sistema debe incrementar las monedas virtuales en tiempo real basándose en el poder de clic del jugador.
RF-03	Producción Pasiva	El sistema debe calcular y sumar automáticamente las monedas generadas por segundo en función del inventario de "Conejos" (empleados) contratados por el usuario.

RF-04	Sistema de Transacciones	El jugador debe poder consultar un catálogo de mejoras (utensilios) y empleados. El sistema debe verificar el saldo disponible en el servidor antes de procesar cualquier compra y actualizar el inventario.
RF-05	Persistencia Asíncrona	El cliente (JS) debe enviar peticiones AJAX (API Fetch) al servidor de forma periódica para guardar el progreso actual en la base de datos sin interferir en la fluidez de la partida.
RF-06	Mecánica de Prestigio	El usuario debe poder reiniciar sus estadísticas a cero a cambio de "Hojas de Té", aplicando una fórmula que otorgue multiplicadores permanentes según el rendimiento histórico de su partida.

Tabla 1. Requisitos funcionales

3.1.2 Requisitos no funcionales

ID	Requisito	Especificación Concreta
RNF-01	Rendimiento y Latencia	Las peticiones asíncronas de guardado en segundo plano deben procesarse y devolver un código HTTP 200 en menos de 100 milisegundos para evitar que el frontend sufra de desincronización ("rubber-banding").
RNF-02	Encriptación de Contraseñas	Las contraseñas de los usuarios deben procesarse mediante funciones de hashing criptográfico seguro (ej. password_hash() de PHP) antes de almacenarse en la base de datos.
RNF-03	Protección en Base de Datos	Todas las consultas SQL deben implementarse obligatoriamente utilizando sentencias preparadas (Prepared Statements) a través de PDO para blindar la aplicación contra inyecciones SQL.

RNF-04	Estructura del Código	El backend debe desarrollarse estrictamente bajo el patrón arquitectónico MVC, separando la capa de presentación (Vistas), la lógica de negocio (Controladores) y el acceso a datos (Modelos).
RNF-05	Adaptabilidad Web	La interfaz debe aplicar técnicas de <i>Responsive Design</i> mediante CSS puro (Grid/Flexbox y Media Queries) para una correcta visualización en cualquier resolución (móvil, tablet, escritorio).

Tabla 2. Requisitos no funcionales

3.2 ESTUDIO DE VIABILIDAD Y DAFO

Antes de programar nada, analicé si realmente podía abarcar este proyecto en los plazos del curso. A nivel económico es totalmente viable porque he optado por herramientas de código abierto gratuitas. A nivel técnico, el reto era grande pero realista.

Para verlo más claro, elaboré este análisis DAFO:

Análisis Interno	Análisis Externo
Debilidades (D) • Curva de aprendizaje pronunciada al diseñar un enrutador y un patrón MVC puro desde cero en PHP. • Falta de un equipo multidisciplinar (diseño gráfico, testing), teniendo que asumir yo todos los roles del desarrollo.	Amenazas (A) • Riesgo de cuellos de botella en la base de datos si muchos jugadores realizan peticiones asíncronas de guardado simultáneamente. • Posibles "condiciones de carrera" (descuadres de saldo) si la latencia de red del usuario es alta durante las compras rápidas.

Fortalezas (F)	Oportunidades (O)
• Control absoluto sobre cada línea de código al no depender de librerías de terceros, evitando el comportamiento de "caja negra". • Arquitectura final extremadamente ligera y rápida, ideal para ejecución en navegadores móviles. • Costes operativos y de licencias inexistentes.	• La lógica relacional y de control de concurrencia sirve como base técnica perfecta para crear aplicaciones empresariales reales (ej. gestión de inventarios). • Posibilidad de escalar el producto a nivel comercial para plataformas como Android o Steam.

Tabla 3. Análisis DAFO

3.3 HERRAMIENTAS DE DISEÑO Y TECNOLOGÍAS

Para el desarrollo de este proyecto he decidido prescindir de *frameworks* y librerías externas. El objetivo de esta decisión ha sido programar la arquitectura desde cero para consolidar los conocimientos técnicos adquiridos. Las tecnologías seleccionadas y su justificación son las siguientes:

- **HTML5 y CSS3:** He diseñado la interfaz de forma personalizada utilizando CSS nativo. Al no depender de librerías de estilos predefinidas como Bootstrap, he aplicado directamente técnicas de *Flexbox* y *CSS Grid* para lograr un diseño *Responsive*. Esto también optimiza el tiempo de carga, ya que el navegador no procesa código innecesario.
- **JavaScript Vanilla:** Toda la interacción del usuario se gestiona con JavaScript nativo. La principal justificación técnica en este punto es el uso intensivo de la **API Fetch**. Mediante peticiones asíncronas (AJAX), el cliente se comunica con el servidor en segundo plano para guardar la partida, logrando una experiencia de juego continua sin recargas de página.

- **PHP 8 (POO):** El *backend* del proyecto está programado íntegramente en PHP aplicando Programación Orientada a Objetos. He implementado un sistema de enrutamiento propio y he estructurado la aplicación bajo el patrón **MVC** (Modelo-Vista-Controlador). De este modo, la capa de acceso a datos queda estrictamente separada de la lógica de presentación.
- **MySQL y PDO:** La persistencia de datos se gestiona en una base de datos relacional MySQL. Para la conexión desde PHP he utilizado la interfaz **PDO**. La elección de PDO frente a otras extensiones se justifica por su seguridad, ya que permite ejecutar sentencias preparadas (*Prepared Statements*) y previene ataques de inyección SQL.
- **Git y GitHub:** He utilizado Git para llevar un control de versiones exhaustivo. Mantener un registro ordenado de los *commits* ha sido fundamental para desarrollar nuevas mecánicas de juego con la garantía de poder revertir el código a una versión estable en caso de error.

3.4 DOCUMENTACIÓN DEL DISEÑO

3.4.1 Diagrama de Casos de Uso

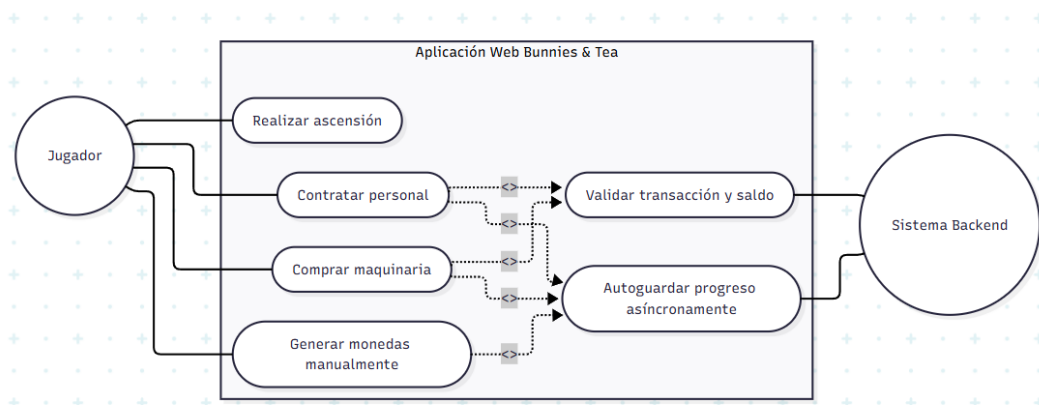


Imagen 1. Diagrama de casos de uso

3.4.2 Modelo Entidad-Relación y Diccionario de Datos

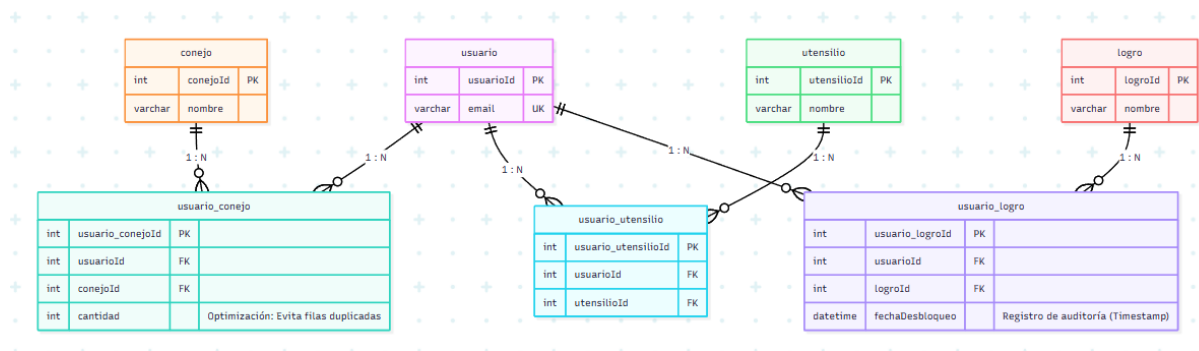


Imagen 2. Modelo entidad-relación

- USUARIO (usuarioId, email, passwordHash, nombreCafeteria, monedasActuales, clicsSucios, ultimoAcceso, monedasHistoricas, puntosLegado)
- CONEJO (conejoId, nombre, rol, produccionBase, costeBase)
- UTENSILIO (utensilioId, nombre, valorExtraClic, produccionPasivaExtra, costeBase, limiteMax)
- LOGRO (logroId, nombre, descripcion, tipoCondicion, valorCondicion, multiplicadorRecompensa)
- USUARIO_CONEJO (usuario_conejoId, usuarioId, conejoId, cantidad)
- USUARIO_UTENSILIO (usuario_utensilioId, usuarioId, utensilioId)
- USUARIO_LOGRO (usuario_logroId, usuarioId, logroId, fechaDesbloqueo)

Tabla usuario Almacena las credenciales y el estado de la partida de cada jugador.

Atributo	Tipo de Dato	Restricciones	Descripción
usuariold	INT	PK, AUTO_INCREMENT	Identificador único del usuario.
email	VARCHAR(255)	NOT NULL, UNIQUE	Correo electrónico usado para el login.
passwordHash	VARCHAR(255)	NOT NULL	Contraseña cifrada mediante algoritmos de hash seguros (bcrypt).
nombreCafeteria	VARCHAR(100)		Nombre personalizable del local del jugador.
monedasActuales	DECIMAL(20,2)	DEFAULT 0.00	Saldo líquido actual del que dispone el usuario.
clicksSucios	DECIMAL(10,2)	DEFAULT 0.00	Nivel de suciedad acumulada por la actividad. Actúa como penalización.
ultimoAcceso	DATETIME	DEFAULT CURRENT_TIMESTAMP	Marca de tiempo para cálculos de desconexión.
monedasHistoricas	DECIMAL(20,2)	DEFAULT 0.00	Total histórico generado. Usado para hitos y prestigio.
puntosLegado	INT	DEFAULT 0	Moneda premium (Hojas de Té) ganada al reiniciar la partida.

Tabla 4. Diccionario de datos de usuario

Tabla conejo Catálogo de entidades que otorgan generación pasiva de recursos (Empleados).

Atributo	Tipo de Dato	Restricciones	Descripción
conejoId	INT	PK, AUTO_INCREMENT	Identificador único del empleado.
nombre	VARCHAR(100)	NOT NULL	Nombre descriptivo del conejo.
rol	VARCHAR(50)		Puesto que ocupa (ej. Camarero, Chef).
produccionBase	DECIMAL(10,2)	NOT NULL	Monedas generadas por segundo (PPS).
costeBase	DECIMAL(15,2)	NOT NULL	Precio de contratación en la tienda.

Tabla 5. Diccionario de datos de conejo

Tabla utensilio Catálogo de mejoras de infraestructura que potencian la generación manual o pasiva.

Atributo	Tipo de Dato	Restricciones	Descripción
utensilioId	INT	PK, AUTO_INCREMENT	Identificador único del utensilio.
nombre	VARCHAR(100)	NOT NULL	Nombre del equipamiento.
valorExtraClic	DECIMAL(10,2)	NOT NULL	Incremento directo al valor de cada clic manual.

produccionPasivaExtra	DECIMAL(10,2)	DEFAULT 0.00	Incremento a la producción pasiva total.
costeBase	DECIMAL(15,2)	NOT NULL	Precio de adquisición.
limiteMax	INT	DEFAULT 1	Cantidad máxima que un usuario puede poseer.

Tabla 6. Diccionario de datos de utensilio

Tabla logro Catálogo de hitos alcanzables dentro del juego.

Atributo	Tipo de Dato	Restricciones	Descripción
logroid	INT	PK, AUTO_INCREMENT	Identificador único del logro.
nombre	VARCHAR(100)	NOT NULL	Título del hito alcanzado.
descripcion	VARCHAR(255)	NOT NULL	Explicación del hito.
tipoCondicion	VARCHAR(50)	NOT NULL	Tipo de trigger (ej. clics, monedas_totales).
valorCondicion	DECIMAL(20,2)	NOT NULL	Umbral necesario para disparar el logro.
multiplicadorRecompensa	DECIMAL(5,2)	DEFAULT 1.00	Bono multiplicador otorgado al desbloquearlo.

Tabla 7. Diccionario de datos de logro

Tabla usuario_conejo (Inventario de Empleados)

Atributo	Tipo de Dato	Restricciones	Descripción
usuario_conejold	INT	PK, AUTO_INCREMENT	Clave subrogada de la relación.
usuariold	INT	FK (usuario.usuariold)	Referencia al jugador.
conejold	INT	FK (conejo.conejold)	Referencia al tipo de empleado contratado.
cantidad	INT	DEFAULT 1	Número de copias de ese mismo empleado.

*Tabla 8. Diccionario de datos de usuario_conejo***Tabla usuario_utensilio (Inventario de Mejoras)**

Atributo	Tipo de Dato	Restricciones	Descripción
usuario_utensiliold	INT	PK, AUTO_INCREMENT	Clave subrogada de la relación.
usuariold	INT	FK (usuario.usuariold)	Referencia al jugador.
utensiliold	INT	FK (utensilio.utensiliold)	Referencia al equipamiento adquirido.

Tabla 9. Diccionario de datos de usuario_utensilio

Tabla usuario_logro (Registro de Hitos Desbloqueados)

Atributo	Tipo de Dato	Restricciones	Descripción
usuario_logroid	INT	PK, AUTO_INCREMENT	Clave subrogada de la relación.
usuarioid	INT	FK (usuario.usuarioid)	Referencia al jugador.
logroid	INT	FK (logro.logroid)	Referencia al logro alcanzado.
fechaDesbloqueo	DATETIME	DEFAULT CURRENT_TIMESTAMP	Momento exacto en que se superó el hito.

Tabla 10. Diccionario de datos de usuario_logro

3.4.3 Mockups e Interfaces



Imagen 3. Mockup de pantalla principal



Imagen 4. Mockup de interfaz de juego



Imagen 5. Vista general de interfaz

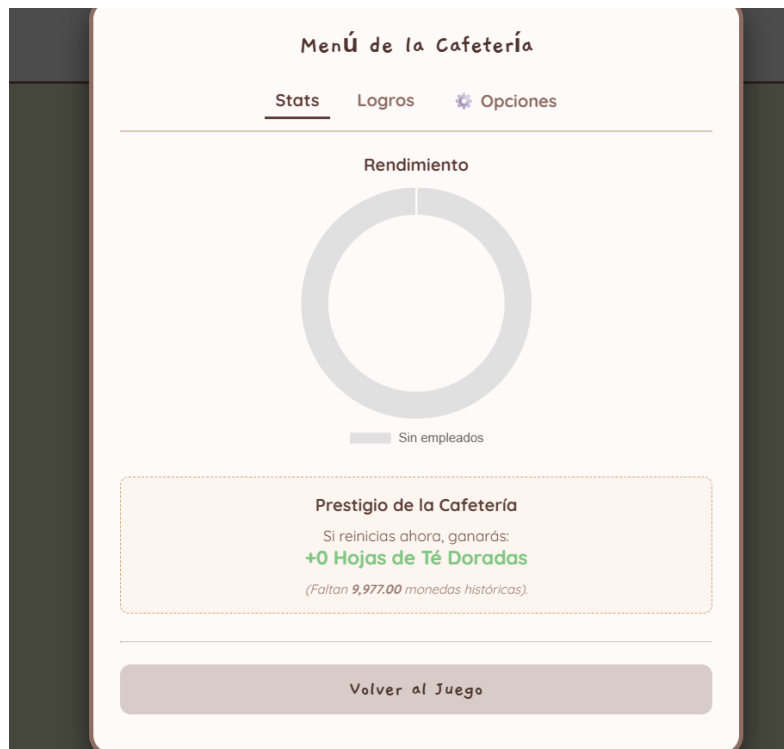


Imagen 6. Mockup de tienda



Imagen 7. Mockup de logros



Imagen 8. Mockup de menú

3.5 FASES DEL PROYECTO

Para garantizar una evolución ordenada del desarrollo, el ciclo de vida del software se dividió en las siguientes fases metodológicas:

- **Análisis:** Fase inicial centrada en la definición de los requisitos funcionales y no funcionales del juego. En esta etapa realicé el estudio de viabilidad y definí las mecánicas principales (generación activa y pasiva de recursos).
- **Diseño:** Etapa dedicada a la arquitectura del sistema. Incluye el diseño del modelo Entidad-Relación de la base de datos, la planificación del patrón MVC para el servidor y la elaboración de los *mockups* para la interfaz de usuario.
- **Codificación:** La fase más extensa del proyecto. Comenzó con la implementación del *backend* (controladores, modelos y enrutador en PHP), seguida de la maquetación *frontend* (HTML/CSS) y finalizando con la lógica del cliente y la comunicación asíncrona mediante JavaScript nativo.
- **Pruebas (Testing):** Durante esta etapa realicé pruebas manuales exhaustivas. Me centré especialmente en comprobar la resistencia del sistema ante "condiciones de carrera" (compras rápidas simultáneas) y en asegurar que la recuperación de la sesión funcionaba correctamente sin pérdida de datos.

- **Mantenimiento y Despliegue:** Fase final orientada a la limpieza y refactorización del código, preparación del entorno de producción y elaboración de esta misma documentación técnica para la presentación del proyecto.

3.6 EVALUACIÓN ECONÓMICA Y FINANCIACIÓN

Aunque se trata de un proyecto académico desarrollado de forma individual, he elaborado un presupuesto simulado para calcular cuál sería el coste real de producción de esta aplicación web si se encargara a nivel profesional.

El presupuesto se divide en costes de personal (horas de desarrollo) y costes de infraestructura (hardware y licencias):

Costes de Personal			
Analista / Diseñador	20 horas	35 € / h	700 €
Programador Backend (PHP/MySQL)	45 horas	30 € / h	1.350 €
Programador Frontend (JS/CSS)	35 horas	25 € / h	875 €
QA (Pruebas de software)	10 horas	20 € / h	200 €
Infraestructura y Licencias			
Servidor VPS (Anual)	1 unidad	120 € / año	120 €
Dominio web (Anual)	1 unidad	15 € / año	15 €
Licencias de Software (IDE, Diseño)	-	0 € (Open Source)	0 €
Presupuesto Total Estimado			3.260 €

Tabla 11. Presupuesto estimado del proyecto

4 PLANIFICACIÓN DEL PROYECTO

4.1 SECUENCIACIÓN DE ACTIVIDADES O TAREAS

Para gestionar el proyecto de manera eficiente, he organizado el desarrollo utilizando una Estructura de Desglose del Trabajo (WBS). Esto me ha permitido dividir el objetivo principal (crear el juego web) en paquetes de trabajo más pequeños y manejables, facilitando la estimación de tiempos y el seguimiento del progreso. A continuación detallo la lista de tareas secuenciadas:

1. Fase de Análisis e Inicio

- Definición de la idea y mecánicas (*Idle/Clicker*).
- Especificación de requisitos (Funcionales y No Funcionales).
- Estudio de viabilidad y análisis DAFO.

2. Fase de Diseño Conceptual y Lógico

- Diseño del Modelo Entidad-Relación y Diccionario de Datos.
- Elaboración del Diagrama de Casos de Uso.
- Diseño de la arquitectura del servidor (patrón MVC).
- Creación de *mockups* y bocetos de la interfaz.

3. Fase de Desarrollo y Codificación

- Configuración del entorno de desarrollo y repositorio (Git).
- Implementación de la Base de Datos (MySQL).
- Programación del Backend (PHP 8):
 - Sistema de enrutamiento y conexión PDO.
 - Controladores de usuarios (Login/Registro).
 - Controladores de guardado asíncrono y tienda.
- Programación del Frontend (HTML5/CSS3/JS):
 - Maquetación *Responsive* de la interfaz.
 - Lógica del *Core Loop* (clics y cálculo de recursos).
 - Implementación de peticiones AJAX (Fetch API).

4. Fase de Pruebas y Depuración

- Pruebas unitarias de las consultas a la BBDD.
- Pruebas de integración entre el frontend y el backend (guardado asíncrono).
- Detección y solución de *bugs* (condiciones de carrera, fallos de sesión).

5. Fase de Cierre y Documentación

- Redacción de la memoria del proyecto (este documento).
- Preparación de la defensa y material audiovisual.
- Despliegue final de la aplicación.

4.2 DIAGRAMA DE SECUENCIACIÓN DE PROYECTO

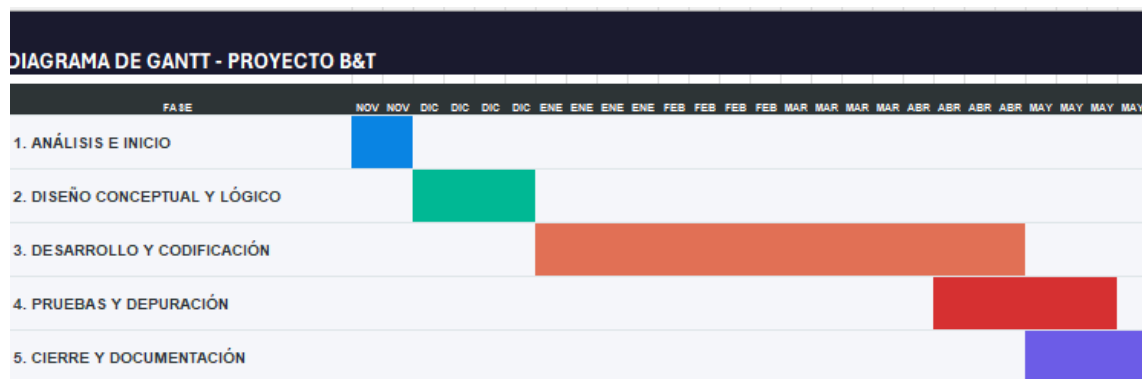


Imagen 9. Diagrama de secuenciación del proyecto

4.3 PROCEDIMIENTOS DE EJECUCIÓN

Para garantizar la estabilidad del código y facilitar el mantenimiento futuro, he establecido un marco de trabajo basado en estándares profesionales de desarrollo de software.

Control de versiones y repositorio Utilizaré **Git** para el control de versiones, realizando subidas periódicas al repositorio remoto. Para que el historial de cambios sea legible y profesional, usaré los siguientes prefijos en los mensajes de cada *commit*:

- **[FEAT]**: Para nuevas funcionalidades
- **[FIX]**: Para correcciones de errores
- **[DOCS]**: Para cambios en la documentación o comentarios en el código.
- **[STYLE]**: Para cambios estéticos de CSS que no afectan a la lógica.

4.4 PREVENCIÓN DE RIESGOS Y SEGURIDAD

En un proyecto de estas características, donde se simula la gestión de recursos de usuarios y guardados en base de datos, la seguridad no podía dejarse en un segundo plano. Para prevenir vulnerabilidades y ataques comunes en entornos web, he implementado una capa de seguridad basada en los siguientes pilares técnicos:

1. **Prevención de Inyección SQL (Uso de PDO)**: He evitado por completo el uso de consultas directas en MySQL mediante funciones obsoletas. Todo el acceso a la base de datos se realiza a través de la interfaz **PDO (PHP Data Objects)**. Esta decisión me ha permitido utilizar *Prepared Statements* (Sentencias Preparadas), lo que significa que todos los datos enviados por el usuario en los formularios de registro o *login* son escapados automáticamente por el sistema. Esto hace matemáticamente imposible que un atacante inyecte código SQL malicioso para manipular la base de datos.

2. **Cifrado de Credenciales (Hashing):** Las contraseñas de los jugadores nunca se guardan en texto plano en la base de datos. He implementado la función nativa `password_hash()` de PHP, que aplica un algoritmo de cifrado robusto. Esto asegura que, incluso en el hipotético caso de que la base de datos fuera vulnerada, las contraseñas de los usuarios estarían protegidas y serían ilegibles. Para el proceso de autenticación, la aplicación utiliza la función correspondiente `password_verify()`.
3. **Protección de Rutas (Middleware de Sesión):** Dado que el juego funciona bajo un patrón MVC estricto, no bastaba con ocultar botones en el HTML. Para evitar que un usuario sin registrar pudiera acceder directamente a la URL de la partida (ej. `/juego`), implementé un *script* de comprobación a modo de *middleware* (`verificar_sesion.php`). Este *script* se ejecuta de forma prioritaria en todos los controladores protegidos, verificando la existencia de la variable de sesión `$_SESSION`. Si el sistema detecta que un usuario intenta saltarse el control de acceso, el *middleware* detiene la carga inmediatamente y lo redirige forzosamente a la página de *login*.

5 IMPLEMENTACIÓN Y CODIFICACIÓN

5.1 ARQUITECTURA DEL SISTEMA (PATRÓN MVC)

Para garantizar que el código sea escalable, mantenible y limpio, el servidor se ha construido siguiendo estrictamente el patrón de diseño **MVC (Modelo-Vista-Controlador)**. Esta arquitectura separa las responsabilidades de la aplicación en tres capas independientes:

- **Modelo (Model):** Representa la capa de datos. Es la única parte de la aplicación que interactúa directamente con la base de datos (mediante sentencias PDO). Su función es recuperar, insertar o actualizar los datos (por ejemplo, el inventario de un usuario).
- **Vista (View):** Representa la interfaz de usuario. Son los archivos HTML/PHP que el usuario visualiza en su navegador. Las vistas no contienen lógica de base de datos; simplemente muestran la información formateada que les pasa el controlador.
- **Controlador (Controller):** Es el cerebro de la aplicación. Recibe las peticiones del usuario (por ejemplo, "comprar un conejo"), pide la información necesaria al Modelo, aplica la lógica de negocio (comprobar si hay saldo suficiente) y decide qué Vista debe cargar para mostrar el resultado.

A nivel de organización de archivos, esta separación se refleja en un árbol de directorios jerarquizado y lógico:

- / **(Raíz del proyecto)**: .gitignore, composer.json, composer.lock, docker-compose.yml, Dockerfile, phpunit.xml, README.md
- /**db**: init.sql
- /**docs**: BUNNIES AND TEA.docx, BUNNIES AND TEA.pdf
- /**src**: cerrar_sesion.php, db.php, index.php, iniciar_sesion.php, obtener_estado.php, registro.php, tienda.php, verificar_sesion.php
- /**src/assets/css**: style.css

- **/src/controllers:** controladorEstadisticas.php, controladorExportacion.php, controladorImportacion.php, controladorLimpieza.php, controladorMonedas.php, controladorProduccionPasiva.php, controladorReinicio.php

- **/src/models:** GestorAutenticacion.php, GestorLogros.php, GestorTienda.php, MotorJuego.php

- **/src/views:** home.view.php, iniciar_sesion.view.php, registro.view.php, tienda.view.php

- **/tests:** BaseTestCase.php, GestorAutenticacionTest.php, GestorTiendaTest.php, MotorJuegoTest.php

5.1.1 Diagrama de clases

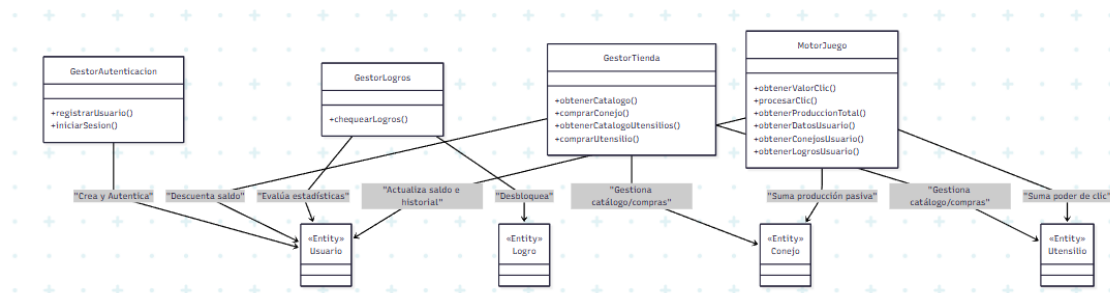


Imagen 10. Diagrama de clases

5.2 DESARROLLO BACKEND

En un simulador del género clicker, la alta frecuencia de interacciones del usuario presenta un desafío crítico a nivel de servidor: el manejo de peticiones concurrentes. Si un jugador realiza múltiples clics en fracciones de segundo, se envían ráfagas de peticiones asíncronas simultáneas. Esto genera un problema clásico de arquitectura conocido como "condición de carrera" (race condition).

Bajo un flujo tradicional de lectura-escritura, el sistema leería el saldo actual, sumaría el valor del clic y guardaría el nuevo resultado. Sin embargo, si dos peticiones llegan exactamente en el mismo milisegundo, ambas podrían leer el valor inicial de 100, procesar la suma por separado y sobrescribir la base de datos con el valor 110 dos veces. El resultado sería una pérdida de datos evidente, contabilizando un solo clic en lugar de dos.

5.2.1 Consultas Atómicas como Solución

Para garantizar la integridad de los datos y asegurar que el servidor mantenga la autoridad absoluta en todo momento, se descartó realizar los cálculos de saldo en la capa de PHP. En su lugar, se delegó la responsabilidad matemática directamente al motor de la base de datos mediante el uso de consultas SQL atómicas.

En la clase MotorJuego.php, en lugar de extraer el dato para manipularlo, se ejecuta una orden directa de actualización.

Mediante este enfoque, es el propio gestor de bases de datos quien bloquea el registro, encola las operaciones y las resuelve de forma secuencial a nivel interno. Esto garantiza que ninguna petición se superponga y que cada clic se sume con absoluta precisión, sin importar cuántos lleguen al mismo tiempo.

5.2.2 Transacciones Seguras (PDO) en la Tienda

Esta misma filosofía de seguridad y blindaje se aplicó en el modelo GestorTienda.php para la adquisición de recursos. Las operaciones de compra, que implican verificar el saldo, restar el coste y añadir el ítem al inventario, se encapsularon dentro de transacciones nativas de PDO.

Esto protege al sistema frente a posibles abusos de latencia (lag). Si un jugador intenta explotar un retraso de red haciendo doble clic en el botón de compra teniendo saldo solo para un ítem, el motor relacional bloquea el proceso, verifica el estado en tiempo real y rechaza la segunda petición, evitando por completo la duplicación de compras o la caída del saldo en números rojos.

5.3 **DESARROLLO FRONTEND**

Para garantizar una experiencia de usuario fluida y evitar la recarga constante del navegador, el cliente web se diseñó para operar de forma asíncrona, acercándose al comportamiento de una Single Page Application (SPA).

Se implementó un bucle principal mediante la función setInterval de JavaScript que, exactamente cada 1000 milisegundos, ejecuta una petición en segundo plano a través de la Fetch API hacia el servidor. Este procesa la producción pasiva generada por el

inventario del usuario, actualiza la base de datos de forma segura y devuelve el nuevo estado al cliente para actualizar la interfaz gráfica.

5.3.1 El problema de la latencia y el "Parpadeo de Red"

En un entorno de interacciones de alta frecuencia (como es el caso de hacer clic rápidamente), la latencia inherente a la red genera un desajuste temporal crítico entre el cliente y el servidor. Cuando el usuario interactúa, el contador local sube inmediatamente (ej. de 100 a 102 en dos clics rápidos), pero la petición tarda unos milisegundos en viajar y ser procesada.

El problema visual (flickering o efecto rubber-banding) ocurre cuando el servidor responde al primer clic indicando el nuevo saldo (101). Si la interfaz gráfica acepta ciegamente ese dato, el contador retrocedería de 102 a 101 visualmente, para luego volver a saltar al recibir la segunda respuesta. Los números saltarían hacia adelante y hacia atrás constantemente.

5.3.2 Predicción del Cliente y Descarte de Respuestas Obsoletas

Para solucionar esta desincronización, se desarrolló un sistema predictivo de control de estado en el Frontend. Se establecieron dos variables de seguimiento: clicsPendientes y peticionesEnVuelo.

Cada vez que JavaScript emite una petición fetch, se incrementa el contador de peticiones en vuelo, restándose cuando se recibe la respuesta. La lógica de renderizado aplica entonces una regla estricta: la interfaz de usuario solo sobrescribe su contador con el total absoluto dictado por el servidor si y solo si peticionesEnVuelo === 0 y clicsPendientes === 0.

Si el cliente recibe datos del servidor pero existen peticiones en vuelo, asume que la respuesta del servidor contiene información "obsoleta" a nivel visual, ya que el usuario ha realizado acciones más recientes que el servidor aún está procesando. En estos casos, el Frontend suma los incrementos correspondientes, pero ignora el total absoluto enviado por el servidor, confiando en su propia predicción local.

Gracias a esta arquitectura, el juego logra una actualización visual suave, instantánea y predictiva, sin comprometer en ningún momento la seguridad ni la autoridad de la base de datos en el servidor.

6 PRUEBAS Y VALIDACIÓN DEL PROYECTO

6.1 EVALUACIÓN Y SEGUIMIENTO

Para garantizar la estabilidad del sistema y asegurar que la economía del juego no sufriera desajustes por errores matemáticos o fallos de base de datos, la validación del proyecto se dividió en dos fases complementarias: pruebas unitarias automatizadas (código) y pruebas manuales de estrés y usabilidad (usuario).

Pruebas Unitarias Automatizadas (PHPUnit) El núcleo lógico del juego debía ser infalible. Un error de cálculo en la producción en segundo plano o en las transacciones invalidaría toda la experiencia. Por ello, se utilizó el *framework* PHPUnit para desarrollar una batería de pruebas sobre los modelos principales (MotorJuego.php y GestorTienda.php).

Se configuró un entorno de pruebas aislado utilizando una base de datos temporal en memoria (SQLite), verificando los siguientes casos críticos:

- **Cálculo de clics y mejoras:** Comprobar que un usuario sin mejoras genera exactamente 1 moneda por clic, y validar que la inserción de un "utensilio" en la base de datos incrementa el valor del clic de forma matemáticamente exacta.
- **Penalización por suciedad:** Simular que la variable clicsSucios supera el límite de 50. El test verifica que la producción total calculada por el servidor se reduce automáticamente a la mitad, tal y como dictan las reglas de negocio.
- **Sistema de prestigio (Legado):** Comprobar que, al disponer de puntos de legado, el multiplicador correspondiente (1.10x, 1.15x, etc.) se aplica de forma correcta sobre la producción base de los empleados.

- **Compras seguras (Rollback):** Validar que el GestorTienda rechaza cualquier intento de compra si el coste es superior al saldo del usuario, asegurando que la transacción de PDO realiza un *Rollback* correcto y mantiene la integridad de los datos.

Pruebas Manuales y Casos Límite Una vez validada la lógica matemática en el servidor de forma aislada, se procedió a realizar pruebas de integración desde la interfaz gráfica, buscando deliberadamente forzar errores y evaluar la robustez del sistema:

- **Prueba de estrés de concurrencia:** Se utilizaron herramientas externas (*autoclickers*) para simular cientos de clics por segundo. Esta prueba demostró el éxito de la arquitectura: la lógica predictiva de peticionesEnVuelo del Frontend evitó el parpadeo visual (*flickering*), mientras que las consultas atómicas SQL garantizaron que el servidor no perdiera el registro de ningún clic.
- **Pruebas de persistencia e interrupciones:** Se forzaron cierres abruptos de la pestaña del navegador durante procesos de compra, recargas bruscas de página y aperturas de sesión simultáneas en distintos navegadores. En todos los casos, el estado al reconectar (conejos, monedas y cálculo de tiempo transcurrido) coincidía de forma exacta y coherente con la base de datos.
- **Reactividad de la Interfaz (UI):** Se comprobó que el *feedback* visual (como los cambios de color y la barra de suciedad al superar los 50 clics) respondía correctamente en tiempo real, guiando al usuario para pagar el mantenimiento y recuperar su tasa de producción pasiva.

6.2 INDICADORES DE CALIDAD

Para validar la robustez del sistema y la base de datos, se definieron y ejecutaron casos de prueba críticos que abarcan tanto la lógica de negocio como la experiencia de usuario.

A continuación se presenta la matriz de resultados:

Caso de Prueba	Condición	Resultado Esperado	Estado
Compra sin fondos suficientes	El usuario intenta comprar un "Conejo Chef" (150 monedas) teniendo solo 100 monedas en el balance.	La transacción SQL se cancela. El servidor devuelve un error y el saldo del usuario se mantiene intacto.	PASS
Doble petición de compra por lag (Abuso)	El usuario tiene 150 monedas. Hace doble clic intencionadamente rápido en "Comprar" para intentar llevarse dos ítems antes de que el servidor descuente el saldo inicial.	La base de datos bloquea el registro en el primer clic. Al entrar el segundo clic, el motor detecta que el saldo ya es insuficiente y la operación se rechaza.	PASS
Latencia alta de red en bucle pasivo	El servidor tarda más de 2 segundos en responder a la petición de producción pasiva, mientras el usuario sigue haciendo clics manuales.	El cliente acumula las monedas localmente mediante predicción y descarta el "saldo total" absoluto que llega con retraso desde el servidor, evitando parpadeos visuales.	PASS
Penalización por suciedad del local	El usuario acumula 50 clics manuales (clicsSucios).	El backend reduce la producción pasiva a la mitad. El frontend cambia la interfaz a estado de alerta (rojo) y habilita el pago de 20 monedas para limpiar la cafetería.	PASS

Tabla 12. Indicadores de calidad

6.3 INFORME DE EVALUACIÓN DE INCIDENCIAS Y SOLUCIONES

Durante las primeras fases de integración, se detectó un fallo visual. Si un jugador hacía clics manuales al mismo tiempo que el bucle de "producción pasiva" recibía datos de fondo, el contador de monedas en pantalla saltaba hacia atrás y hacia adelante. Esto ocurría porque la interfaz aceptaba ciegamente la respuesta del servidor que llegaba con latencia, sobrescribiendo y borrando los clics locales. Se reestructuró la arquitectura de renderizado del cliente. En lugar de que el servidor dicte el número absoluto en cada actualización, se implementó un sistema de Predicción del Cliente. Mediante el uso de variables de control de red (peticionesEnVuelo y clicsPendientes), el navegador confía temporalmente en sus propios cálculos matemáticos. Solo cuando el canal de red está completamente libre (sin peticiones encoladas), el cliente se resincroniza con el estado definitivo validado por el servidor. Esta lógica solucionó el 100% de los parpadeos.

6.4 GESTIÓN DE CAMBIOS

Ha habido tres modificaciones arquitectónicas y de diseño clave respecto al planteamiento original, orientadas a mejorar la retención del jugador y el rendimiento técnico:

- **Transición a comportamiento SPA (Single Page Application):** Originalmente, las interacciones web requerían recargas de página o bloqueaban la interfaz. Se decidió refactorizar el núcleo hacia un bucle asíncrono puro (*Fetch API* cada 1000ms) para ofrecer un flujo continuo y en tiempo real.
- **Mecánica de "Suciedad" (Freno económico):** En las primeras pruebas, el juego resultaba demasiado pasivo. Para forzar una interacción activa del jugador y crear un sumidero de monedas que balanceara la economía, se añadió el sistema de "clics sucios". Esto obliga al usuario a gastar recursos en limpieza para mantener su producción al 100%.
- **Sistema de Prestigio (Puntos de Legado):** El prototipo inicial tenía un final abrupto al adquirir todas las mejoras. Para dotarlo de rejugabilidad infinita, se desarrolló una mecánica de reinicio, donde el usuario sacrifica su partida actual a cambio de

ganar multiplicadores permanentes, añadiendo una nueva capa de estrategia a largo plazo.

7 IMPLANTACIÓN DEL PROYECTO

7.1 PLAN DE IMPLANTACIÓN Y DESPLIEGUE

Para garantizar que el proyecto sea fácilmente escalable, reproducible y evitar el clásico problema de *"en mi máquina sí funciona"*, se ha descartado la instalación manual tradicional de servidores web (como XAMPP o WAMP). En su lugar, el despliegue del proyecto se ha orquestado utilizando contenedores Docker.

Se ha creado un archivo `docker-compose.yml` que define toda la infraestructura como código (*Infrastructure as Code*). Este archivo levanta simultáneamente tres servicios interconectados en una red virtual aislada:

- **app:** Un contenedor con el servidor web (Apache) y PHP 8, que expone el juego en el puerto 8080.
- **db:** Un motor de base de datos MariaDB. Al levantarse por primera vez, lee automáticamente el archivo `init.sql` de la carpeta `/db` para crear las tablas relacionales y rellenar los datos iniciales del catálogo sin intervención humana.
- **phpmyadmin:** Una interfaz gráfica de administración de base de datos accesible en el puerto 8081 para facilitar tareas de mantenimiento y auditoría.

El uso de esta tecnología moderna de despliegue permite que cualquier persona, independientemente de su sistema operativo, pueda levantar toda la arquitectura de Backend y Frontend con un solo comando en cuestión de segundos, garantizando un entorno idéntico al de desarrollo.

7.2 MANUAL DE INSTALACIÓN

Gracias a docker, los requisitos previos para instalar el juego son mínimos. Solo es necesario tener instalados **Git** y **Docker Desktop** (o Docker Engine) en la máquina anfitriona.

7.2.1 Pasos de instalación:

1. Clonar el repositorio del proyecto en el equipo local abriendo la terminal y ejecutando:

```
git clone [URL_DEL_REPOSITORIO]
```

2. Acceder al directorio raíz del proyecto clonado:

```
cd BaT
```

3. Construir y levantar los contenedores en segundo plano usando Docker Compose:

```
docker-compose up -d --build
```

Una vez finalizado el proceso, el juego estará funcionando. Para acceder a él, solo hay que abrir cualquier navegador web e introducir la dirección: <http://localhost:8080>. Si se desea gestionar la base de datos de forma visual, se puede acceder a phpMyAdmin en <http://localhost:8081>.

7.3 MANUAL DE USUARIO

Bunnies & Tea es un juego de tipo incremental (*clicker*). La interfaz está diseñada para ser intuitiva desde el primer momento:

- **Cómo jugar:** El objetivo principal es ganar monedas para mejorar tu cafetería. Puedes generar dinero de dos formas: activamente (haciendo clic manual en el botón principal "Servir Té") o pasivamente. Con las monedas ganadas, debes acceder a la pestaña "Tienda" en el menú de navegación para contratar conejos (que generan dinero cada segundo de forma automática) o comprar utensilios (que aumentan el valor base de tus clics manuales).

- **Mecánica de penalización (Limpieza):** Al hacer demasiados clics manuales, la barra de suciedad de la cafetería irá subiendo. Si supera los 50 *clics sucios*, el local entrará en estado crítico y la producción pasiva de todos tus empleados bajará a la mitad. El jugador debe pulsar el botón "Limpiar" y pagar una pequeña penalización económica para recuperar su nivel de ingresos óptimo.
- **Opciones Avanzadas (Panel de Administración):** En el menú superior derecho (icono de engranaje ) se encuentra el panel de gestión de la partida:
 - **Reiniciar partida (Sistema de Prestigio):** Si un jugador ha acumulado suficientes monedas históricas, podrá realizar un "Reinicio". Esto borra sus empleados y monedas actuales, pero le otorga "Hojas de Té Doradas" permanentes. Estas hojas le darán un bonus multiplicador de producción para su próxima partida, permitiéndole avanzar mucho más rápido.
 - **Exportar / Importar partida:** Para no perder el progreso si el contenedor se apaga o se cambia de ordenador, el jugador puede ir a " Opciones" y pulsar en "Descargar Partida (.json)". Esto generará un archivo cifrado con todos sus progresos y logros. En la pantalla de inicio de sesión (antes de entrar al juego), existe un botón para subir este archivo JSON y restaurar la partida exactamente donde la dejó.

8 CONCLUSIONES

8.1 SÍNTESIS DE RESULTADOS

Tras la finalización del desarrollo, se puede afirmar que los objetivos principales del proyecto se han cumplido con éxito y dentro de los plazos estipulados. Se ha logrado construir un videojuego web de tipo clicker plenamente funcional, con una economía escalable, persistencia de datos segura y un sistema de renderizado asíncrono que ofrece una experiencia de usuario (UX) fluida.

Los problemas arquitectónicos inherentes a este tipo de aplicaciones de alta frecuencia (como las condiciones de carrera o el parpadeo de red) fueron identificados a tiempo y resueltos implementando patrones de diseño avanzados, lo que demuestra la viabilidad y robustez técnica del producto final.

8.2 LÍNEAS FUTURAS DE TRABAJO

Aunque el producto actual es estable, el proyecto ha sido diseñado con una arquitectura modular que permite una fácil expansión hacia una futura Versión 1.0

Las principales líneas de trabajo se dividirían en tres pilares:

- Escalabilidad técnica extrema (Batching y Client-Side Prediction): Actualmente el juego es robusto para un volumen moderado de usuarios. Sin embargo, para soportar a miles de jugadores concurrentes sin saturar la base de datos relacional, se llevaría la "Predicción del Cliente" al siguiente nivel. En lugar de informar al servidor cada pocos segundos, el motor matemático se ejecutaría casi al 100% en el navegador del jugador. El cliente encolaría todas las interacciones, clics y ganancias pasivas localmente, y enviaría un único "paquete de sincronización cifrado" al servidor cada 30 segundos (Batching). El backend actuaría puramente como validador de trampas y guardado final, reduciendo las consultas SQL y el consumo de CPU del servidor en más de un 95%.
- Profundidad de Game Design y Decisiones: A nivel de jugabilidad, el objetivo de la Versión 1.0 es dotar a la cafetería de más personalidad y opciones estratégicas. Se implementarían trade-offs o sacrificios (ej. un objeto que multiplica enormemente tus ingresos pero ensucia el local el triple de rápido). También se planea incluir

pequeños minijuegos de atención al cliente para romper con la monotonía del formato idle, obligando al jugador a tomar decisiones que definan el rumbo de su partida.

- Salto de calidad visual: El diseño de interfaz actual es limpio y funcional, pero el atractivo principal de un simulador de gestión relajante debe ser su apartado artístico. Se sustituiría el arte actual por un diseño con ilustraciones profesionales, de alta calidad y estética cozy, añadiendo animaciones complejas a los conejos interactuando con la cafetería, lo que aumentaría dramáticamente la retención del usuario.

8.3 CONCLUSIÓN PERSONAL

A nivel técnico, desarrollar este proyecto ha sido un reto enorme y me ha permitido dar un gran salto como programadora. Crear una aplicación de este tipo me ha obligado a ir más allá del desarrollo web convencional y enfrentarme a problemas propios de un entorno real: gestionar peticiones concurrentes, exprimir al máximo la Fetch API, diseñar consultas SQL atómicas para garantizar la integridad de los datos y desplegar toda la infraestructura mediante contenedores Docker. Ahora tengo muy claro que programar no consiste solo en lograr que el código funcione localmente, sino en construir un sistema robusto, a prueba de manipulaciones y capaz de mantener el rendimiento en el servidor.

A nivel personal, este proyecto me ha enseñado lecciones muy valiosas sobre cómo gestionar la frustración y la importancia de la organización. Cuando te encuentras con un error complejo que no tiene una solución evidente en internet, no queda más remedio que pensar de forma analítica: dividir un problema grande en partes manejables y estructurar muy bien la lógica antes de empezar a programar. Ha sido un proceso exigente y con momentos de mucha tensión, pero ver ahora el juego funcionando de forma fluida y empaquetado en su propio contenedor compensa todo el esfuerzo. Terminé este proyecto con una enorme satisfacción y con una confianza técnica mucho mayor para enfrentarme a mi futuro laboral.

9 BIBLIOGRAFÍA Y WEBGRAFÍA

Para el diseño, desarrollo e implantación de este proyecto, se han consultado de forma extensiva las documentaciones oficiales de las tecnologías empleadas, así como recursos educativos de ingeniería de software:

- Documentación Oficial de PHP (The PHP Group): Manual de PHP, Referencia de Objetos de Datos de PHP (PDO), manejo seguro de contraseñas (password_hash) y control de sesiones.
 - Disponible en: <https://www.php.net/manual/es/>
- MDN Web Docs (Mozilla Developer Network): Referencia de JavaScript moderno (ES6+), implementación de promesas y comunicación asíncrona mediante la Fetch API.
 - Disponible en: <https://developer.mozilla.org/es/>
- Documentación de MariaDB (MariaDB Foundation): MariaDB Knowledge Base, Sintaxis SQL, atomicidad y control de Transacciones (BEGIN, COMMIT, ROLLBACK).
 - Disponible en: <https://mariadb.com/kb/en/>
- Documentación de Docker (Docker Inc.): Referencia de Docker Desktop y orquestación de contenedores mediante Docker Compose.
 - Disponible en: <https://docs.docker.com/>
- Documentación de Chart.js: Guías de implementación de gráficos dinámicos (Doughnut Charts) mediante elementos Canvas de HTML5.
 - Disponible en: <https://www.chartjs.org/docs/latest/>
- W3C (World Wide Web Consortium): Estándares de HTML5 y CSS3, con especial énfasis en el diseño responsivo usando Flexbox y Media Queries.
 - Disponible en: <https://www.w3.org/>